



C1

What is an agent?

~45 min · the loop · memory · patterns · when not to build one

BY THE END OF THIS CAPSULE YOU WILL BE ABLE TO

Tell an agent apart from a plain prompt, a workflow and a RAG pipeline — name which one your task actually needs, and say what the difference costs.

So — what is an agent?

DEFINITION

One definition from the reading — and the two things that must be true for it to hold.

The definition. Anthropic, in Pre-read 1: an agent is a system where the model “dynamically directs its own processes and tool usage, maintaining control over how it accomplishes tasks.”

In one line: an agent decides its own next step, takes it, looks at what happened, and decides again — until it decides to stop.

Two conditions — take either away and it is something else. (1) The steps are not known in advance — the model chooses them at runtime, so their number varies with the input. (2) It gets ground truth back at every step — a tool result, a query, code that ran — so reality can correct it. Without (1): a workflow. Without (2): a monologue.

Three tasks, three answers: “Translate this page” → one call. “Extract invoice fields, validate, post to the ledger” → a workflow: the same three steps every time (opening quiz, Q1). “Find out why this shipment is late and get a new date” → an agent: nobody can say how many steps that takes.

Everything after this slide is a consequence of those two conditions — the loop, the memory, the patterns, and the reasons it fails.

Takeaway Two tests: the model picks the steps, and the world answers back. Both, or it is not an agent.

You have already used one

GROUP A • FAST PASS

Four systems most of this room has met. Each one is an agent by the two tests — most people do not call them that.

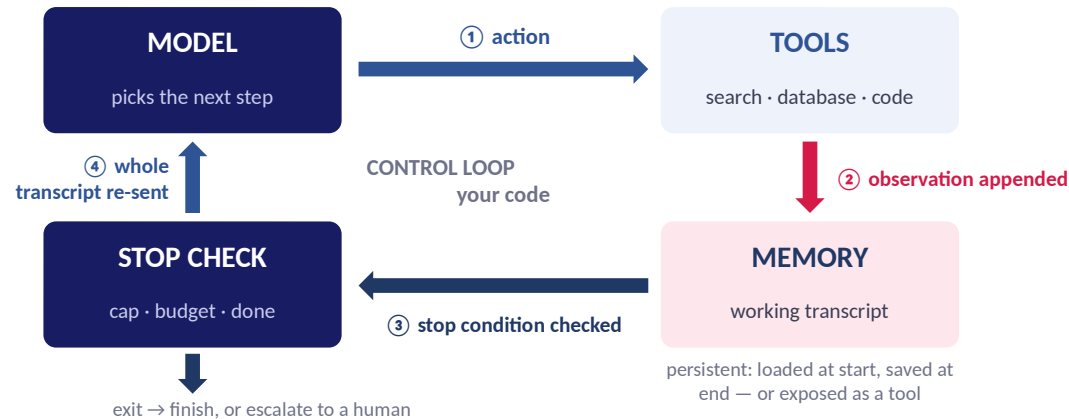
Where you have met it	What it actually does	Why it is an agent
“Deep research” in a chat assistant	plans a set of searches, reads what comes back, notices what is missing, searches again, then writes with citations	the number of searches depends on what it finds — nobody wrote that sequence
An AI coding assistant asked to fix a failing test	reads the repo, runs the test suite, edits, re-runs — until it is green or it gives up	the test suite is ground truth, and it arrives in seconds (Pre-read 5)
Google’s Gemini Spark	watches mail and calendar, chains steps across apps, and asks before it spends your money	always-on, steps chosen at runtime, with a human gate at the irreversible step (Pre-read 4)
The support assistant that actually rebooks you	looks up the booking, checks the fare rules, finds an alternative, makes the change	it acts — unlike the FAQ bot beside it, which only retrieves and answers

Takeaway Agents are already ordinary. The word is new; the pattern is in products you use weekly.

From one call to a loop

An agent is four parts, a stop button, and orchestration around the lot. The loop is yours — you write it, not the model.

ORCHESTRATION what runs, in what order, and what happens on failure



Every arrow is your code. The model produces only ① — everything else is orchestration.

- **1 • Model** — the reasoning engine.
- **2 • Tools** — what it may call: name, description, typed arguments.
- **3 • Memory** — holds this run; keeps across runs.
- **4 • Control loop** — your code: call, execute, append, repeat.
- **5 • Stop condition** — cap, budget or “done”. Never optional.
- **6 • Orchestration** — which model, which path, what happens on failure.

Takeaway An agent = model + tools + memory, wrapped in a loop that YOU control and YOU stop.

MECHANISM

The loop, as text

Everything an agent “does” is tokens in a growing transcript — except one line.

A real trajectory. **Thought:** “I need this supplier’s lead time — look it up rather than guess.” → **Action:** lookup_supplier("SG-4471") → **Observation:** {"lead_time_days": 21} → **Thought:** “That misses the build date. Check the alternate.” → **Action:** lookup_supplier("SG-8802") → **Observation:** {"lead_time_days": 9} → **Action:** finish("Switch to SG-8802.")

Notice: the model wrote every line except the Observations — the red ones. Those came from your code, running a real query.

Why that one line matters. The Observation is the only point where the outside world can contradict the model. Remove it and you have chain-of-thought: fluent, self-consistent and unfalsifiable. It is also the most expensive part of the loop, because whatever a tool returns is pasted into the transcript and resent on every later turn.

For builders: return the number, not the web page. Observation size is simultaneously an accuracy decision and a cost decision.

Pre-read 2 — Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models” (2022).

Takeaway The Observation is the only door to reality — and the biggest line on the bill.

What counts as a tool?

GROUP A • FAST PASS

Sort tools by what they do to the world — the same five kinds show up in every industry.

What it does	Generic examples	In your industry	Undo?
Retrieve — read something	web search, inbox search, vector store	case-law search, patient record lookup, CRM fetch	yes
Compute — transform, deterministically	calculator, code execution, spreadsheet	dosage calculator, pricing engine, tax rules	yes
Act on a system — write	create a ticket, update a record	place a lab order, submit a claim, issue a quote	NO
Communicate — write to a person	send an email, post to chat, notify	message the pharmacist, email the client	NO
Sense — read the physical world	OCR, transcription, image analysis	scan-report parsing, meter reading	yes

Rows 3 and 4 are where every governance question in this course lives — approvals, audit trails, and the guest session. Rows 1, 2 and 5 only look. Google's Spark ships exactly this way — permissions off by default, and it asks before it spends your money (Pre-read 4).

Takeaway A tool that only reads is a research assistant. A tool that writes makes the system an actor.

Memory — what it holds, and what it keeps

DEFINITION

Two different things share one word. Confusing them is how agent projects overspend.

Working memory — the transcript itself. Every thought, action and observation so far, resent in full on each turn. It is free to write and it is the reason a long run costs more per turn than a short one. It also degrades: as the window fills, the model attends less well to any one thing in it — Anthropic calls this context rot.

In one line: working memory is what the agent is holding.

Persistent memory — a store outside the run. A notes file, a progress log, a database row, a vector store. The agent reads it at the start and writes to it at the end, so tomorrow's run knows what today's did. This is the difference between an assistant that resumes and a chat that ends.

In one line: persistent memory is what the agent chose to keep — and, in a regulated setting, what you must be able to explain.

Capsule 2 takes this further: compaction, note-taking and sub-agent summaries as ways of managing both.

Takeaway Holding is not keeping. Decide deliberately which one your agent needs.

Who decides the control flow

Decided in advance by you, or in the moment by the model — that, not sophistication, is the line.

The question	Workflow	Agent
Who decides the sequence of steps, and when?	decided in advance, by you, in code	decided in the moment, by the model
Does the number of steps vary with the input?	no — the same path every time	yes — that is the whole point
Can you test every path?	yes, they are enumerable	no — you test outcomes, not paths
What does it cost?	predictable: n calls	unpredictable until you cap it

Anthropic's definitions: *workflows orchestrate models “through predefined code paths”; agents “dynamically direct their own processes and tool usage.”*

Takeaway If the number of steps doesn't vary with the input, you want a workflow — however clever the pitch sounds.

The chooser — seven rungs, cheapest first

Every rung below the first buys flexibility and pays in tokens, latency and predictability.

Rung	Who decides	Cost	Use it when
1 • Single call	you	1 call	one-shot, well-specified task
2 • Prompt chain	you	2–4 calls	fixed subtasks with a check between
3 • Routing	model picks a lane	1 + 1 calls	distinct categories; cheap model for easy ones
4 • Parallelisation	you	N calls, one wall-clock	independent checks, or a vote
5 • Orchestrator-workers	model, at runtime	unpredictable	the split depends on the input
6 • Evaluator-optimiser	model	unknown rounds	criteria statable; a second pass helps
7 • Agent	model	unbounded until capped	steps cannot be known in advance

Rungs 1–6 are workflows: predictable and testable. Only rung 7 hands over the sequence. • [Pre-read 1 — Anthropic](#).

Takeaway Go down the list only when the rung above genuinely cannot do the job.

Three agent patterns

GROUP A • FAST PASS

Same loop underneath. What changes is where the planning happens — and how it fails.

Pattern	How it works	How it fails
ReAct	one loop: think → act → observe → repeat, deciding the next step each time	repeats itself, or wanders when observations are poor
Planner-executor	plan the whole sequence first, then execute it step by step	the plan meets reality and nothing re-plans
Multi-agent	specialised agents with a coordinator handing out sub-tasks	parallel agents make conflicting assumptions nobody reconciles

You will build the first one after the break. The other two are variations on where planning sits, not different machines. Repetition and an unrevised plan are the same absence — nothing predicts what an action will change (Pre-read 3).

Takeaway Pick the pattern for how the task fails, not for how advanced it sounds.

Multi-agent, honestly

THE CORRECTION

One company argued both sides in ten months — and the second argument is better than the first.

June 2025 — “Don’t build multi-agents.” Two principles: share the full trace, not single messages; and actions carry implicit decisions, so conflicting decisions produce incoherent results. Their example: a Flappy Bird clone in which one sub-agent builds a Mario-style background while another builds a mismatched bird, and nothing can reconcile them.

Why it happened: parallel agents cannot see each other’s assumptions, so each quietly decides something the others contradict.

April 2026 — “Multi-agents: what’s actually working.” Not a reversal, a narrowing: several agents may contribute intelligence, but the writes stay single-threaded. What works — a review agent with clean context reading another agent’s output (~2 bugs per pull request, 58% severe), and a cheap model escalating hard problems to a frontier one.

The rule to keep: multiple minds, one hand on the keyboard.

Source: Cognition engineering blog, Jun 2025 and Apr 2026 (Pre-read 5 covers the same company’s arc).

Takeaway Split the thinking if you like. Do not split the writing.

RAG, agentic RAG, or an agent?

Three systems people call by each other's names. Three questions separate them cleanly.

	Who picks what to retrieve?	Can it loop and re-query?	Can it change the world?
RAG	your code — one fixed query	no — a single pass	no — read-only
Agentic RAG	the model, at runtime	yes — it re-queries on what it finds	no — read-only
Agent	the model, at runtime	yes	YES — it writes, sends, books, pays

So: agentic RAG is simply an agent whose tools are all read-only. The governance cliff is not RAG → agentic RAG. It is agentic RAG → agent, at the first write.

Takeaway Retrieval makes it better informed. Writing makes it dangerous.

One patient, three systems

Same clinical question, escalating ask. You built row 1 yourselves in Class 2.

The ask	What the system does	Where it stops
RAG — “What does our formulary say about metformin in renal impairment?”	one retrieval over the guideline corpus, then one answer with a citation	at the first answer — it cannot check THIS patient’s kidney function

Illustration of system design, not clinical guidance.

Takeaway Ask what the system may DO, not how clever it sounds.

One patient, three systems

Same clinical question, escalating ask. You built row 1 yourselves in Class 2.

The ask	What the system does	Where it stops
RAG — “What does our formulary say about metformin in renal impairment?”	one retrieval over the guideline corpus, then one answer with a citation	at the first answer — it cannot check THIS patient’s kidney function
Agentic RAG — “Is this patient’s metformin dose safe?”	queries the guideline → notices the answer depends on renal function → queries the lab results → finds eGFR 38 → re-queries the guideline for that band → answers, with both sources cited	when the evidence is sufficient. Several retrievals — still read-only

Illustration of system design, not clinical guidance.

Takeaway Ask what the system may DO, not how clever it sounds.

One patient, three systems

Same clinical question, escalating ask. You built row 1 yourselves in Class 2.

The ask	What the system does	Where it stops
RAG — “What does our formulary say about metformin in renal impairment?”	one retrieval over the guideline corpus, then one answer with a citation	at the first answer — it cannot check THIS patient’s kidney function
Agentic RAG — “Is this patient’s metformin dose safe?”	queries the guideline → notices the answer depends on renal function → queries the lab results → finds eGFR 38 → re-queries the guideline for that band → answers, with both sources cited	when the evidence is sufficient. Several retrievals — still read-only
Agent — “Sort out this patient’s metformin.”	all of the above, then drafts the reduced dose in the record, books a repeat renal panel in three months, and messages the pharmacist	when the TASK is done — and three of those steps are hard to undo

Illustration of system design, not clinical guidance. • Rows 1–2 fail by being unhelpful; row 3 fails by being wrong in public.

Takeaway The jump that matters is the last one: the first write turns a research assistant into an actor.

When NOT to build an agent

THE HARD PART

Two tests. If a task fails either one, build the rung below instead.

1 • The ground-truth test. Ask first: what will tell this loop it is wrong, and how fast? A failing test suite answers in seconds and cannot be argued with; a customer's satisfaction answers in days and is a matter of opinion. Fast and objective → build the agent, wiring that signal in first. Slow or subjective → a workflow with a human gate.

And if there is no signal at all: you do not have an AI problem, you have an unspecified one — no amount of autonomy fixes an objective nobody can state.

2 • The arithmetic. A step that works 95% of the time, run 20 times: $0.95^{20} \approx 0.36$ — roughly a one-in-three chance of a clean run. At 99% per step it is still only ≈ 0.82 . Three ways out: raise per-step reliability (hardest), cut the number of steps (biggest win), or make failure recoverable with gates, checkpoints and retries.

Which is why: most of a good agent, by volume of code, is recovery machinery rather than intelligence.

Quiz question 6, in the opening — the number surprises almost everyone, including people who build these for a living. And autonomy is a dial, not a switch: suggest → confirm → act. You set that dial in code after the break (Pre-read 5).

Takeaway Autonomy without recovery scales badly. Fewer steps beats smarter steps.

Same pattern, different industries

GROUP A • SELF-READ

Four real tasks. Notice how few of them actually need rung 7.

Sector	The task	The rung it really needs
Bank	reconciling a failed payment across three systems	agentic RAG — and an agent only once it may post the correction
Hospital	chasing a prior-authorisation through several payer portals	agent — it must submit, wait, and resubmit on what comes back
Manufacturer	chasing a delayed shipment with a supplier	agent with exactly one write: sending the email
Telco	triaging an outage alert to the right team	routing — a workflow, not an agent

*Read the third column as a ladder: only two of these four are truly rung 7, and one of those needs **exactly one write**. The bank row is the governance argument in a single line — same reasoning, same tools, and everything changes the moment it may post.*

Takeaway Most “agent” projects are one rung lower than the pitch deck says — and better for it.

A task in your work an agent could own.

1 · Name the task — and who does it today

2 · Run the three questions: who retrieves, can it loop, can it write

3 · What is the first irreversible action?

4 · Which rung would you actually build?

6 minutes in pairs · we will take three · keep it to a task you have actually seen done.

CAPSULE 1 • TAKEAWAY

An agent is a loop that decides.

Everything else — patterns, memory, retrieval — is a choice about who decides the next step and what the system is allowed to touch. Next (Capsule 2): give it one tool, watch it break, and put the guardrails on.